

Specifiche Algebriche

Marco Alberti



**Università
degli Studi
di Ferrara**

Ingegneria del Software Avanzata
CdLM in Ingegneria Informatica e dell'Automazione
A.A. 2025-2026

Ultima modifica: 30 marzo 2026

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright.
Ne sono vietati la riproduzione e il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore.

Sommario

- Strutture algebriche
- Specifiche algebriche
- Esempi (in linguaggio CASL)
- Dimostrazione di proprietà
- Implementazione
- Problemi in specifiche algebriche
- Valutazione

Strutture algebriche

Una **struttura algebrica** è definita da un **insieme** di **elementi** e da alcune **operazioni** con certe **proprietà**.

Esempio

L'insieme $\mathbb{N} = \{0, 1, \dots\}$ con l'operazione $+: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ è una struttura algebrica detta **monoide commutativo** perché soddisfa queste proprietà:

- $\forall a, b, c \in \mathbb{N} \ (a + b) + c = a + (b + c)$ (associatività)
- $\forall a, b \in \mathbb{N} \ a + b = b + a$ (commutatività)
- $\exists e \in \mathbb{N} \ (\forall a \in \mathbb{N} \ e + a = a + e = a)$ (elemento neutro; quell' e è detto 0)

L'insieme $\mathbb{N}_0 = \{1, 2, \dots\}$ non è un monoide con l'addizione; ma lo è con la moltiplicazione (con elemento neutro 1).

Esempio

L'insieme $\mathbb{R}$ dei numeri reali con $+$ e $\times$ è una struttura algebrica detta **campo**.

Specifica algebrica

Una specifica algebrica di un sistema, o di parte di esso, è composta da:

- uno o più tipi di dato (**sort**):
saranno implementati con i costrutti per la definizione e l'astrazione sui dati resi disponibili dal linguaggio utilizzato (tipi primitivi e derivati, classi, ...)

Esempio: `Lista`, `Pila`, `Nat`

- **operazioni** (funzioni) su dati dei sort dichiarati

Esempio:

- `lunghezza`: `Lista` $\rightarrow$ `Nat`
- `pop`: `Pila` $\rightarrow$ `Nat` $\times$ `Pila`

- **assiomi** che definiscono le proprietà essenziali delle operazioni

Esempio (sintassi Haskell):

- `lunghezza([]) = 0`
- `lunghezza(_:t) = 1 + lunghezza(t)`

Linguaggi e strumenti

Riferimento:

<https://www.informatik.uni-bremen.de/cofi/Language/index.html>

Linguaggi:

- Larch
- CASL (CoFI)
- linguaggi specifici per semantica e sistemi modellati
- non vedremo sintassi formale
- nostra semantica: multi-sorted first order logic

Strumenti:

- analizzatori e dimostratori di proprietà
- molti disponibili in "Heterogeneous Tool Set":
 - home page: <http://hets.eu/>
 - interfaccia web: <http://rest.hets.eu/>

Esempio: Liste con inserimento ordinato (linguaggio CASL)

160_specifiche_algebriche/list.casl

```
1 spec LIST [sort Elem pred le: Elem * Elem] =
2   generated type
3     List ::= nil | cons(Elem;List)
4   ops
5     ins: Elem * List -> List
6   preds
7     sorted: List
8   vars e,h:Elem; l,t:List
9     . ins(e,nil) = cons(e,nil);
10    . le(e,h) => ins(e,cons(h,t)) = cons(e,cons(h,t));
11    . not le(e,h) => ins(e,cons(h,t)) = cons(h,ins(e,t));
12    . sorted(nil);
13    . sorted(cons(e,nil));
14    . sorted(cons(e,cons(h,t))) <=> le(e,h) /\ sorted(cons(h,t))
15 end
```

Dimostrazione di proprietà

Quali proprietà hanno le operazioni, come caratterizzate dalla specifica?

Esempio

Dimostriamo che, per ogni $e:Elem$ e $l:List$, $sorted(l) \Rightarrow sorted(ins(e,l))$, cioè che l'inserimento mantiene l'ordinamento, per induzione sul numero di elementi di l (cioè il numero di applicazioni di $cons$ necessarie a costruirla).

- se l ha 0 elementi, $l=nil$, allora dall'assioma $9\ ins(e,l)=cons(e,nil)$ che è $sorted$ per l'assioma 13.
- supponiamo che la tesi valga per ogni l con $n - 1$ elementi e dimostriamo che vale per l di n elementi. In questo caso, $l=cons(h,t)$, e ci sono due casi:
 - se $le(e,h)$, per l'assioma $10\ ins(e,l)=cons(e,cons(h,t))$ e si applica direttamente l'assioma 14
 - altrimenti, per l'assioma $11\ ins(e,l)=cons(h,ins(e,t))$. Poiché $sorted(cons(h,t))$, $sorted(t)$, quindi $sorted(ins(e,t))$ per l'ipotesi induttiva. $sorted(ins(e,t))$ ha come primo elemento e , oppure il primo elemento di t , e siccome h è minore o uguale a entrambi si può applicare l'assioma 14.

Esempio: implementazione Haskell

160_specifiche_algebriche/list1.hs

```
ins :: Ord a => (a,[a]) -> [a]
ins(e,[]) = [e]
ins(e,h:l) =
    if e <= h
    then e:(h:l)
    else h:(ins(e,l))

sorted :: Ord a => [a] -> Bool
sorted [] = True
sorted [_] = True
sorted (e:(h:t)) = e <= h && sorted(h:t)
```


Esempio: implementazione Haskell

160_specifiche_algebriche/list2.hs

```
data Lista el = Nil | Cons el (Lista el)

ins :: Ord el => el -> Lista el -> Lista el
ins e Nil = Cons e Nil
ins e (Cons h t) =
    if e <= h
    then Cons e (Cons h t)
    else Cons h (ins e t)

sorted :: Ord el => (Lista el) -> Bool
sorted Nil = True
sorted (Cons _ Nil) = True
sorted (Cons e (Cons h t)) = e <= h && sorted (Cons h t)
```

Esempio: stringhe (linguaggio CASL)

160_specifiche_algebriche/string.casl

```
1 spec STRING = Nat AND Char
2   then
3     generated type
4       String ::= new | add(String; Char)
5     ops
6       append: String * String -> String
7       length: String -> Nat
8     preds
9       isEmpty: String
10      equal: String * String
11    vars s, s1, s2: String; c: Char
12      . isEmpty(new);
13      . not isEmpty(add(s, c));
14      . length(new) = 0;
15      . length(add(s, c)) = length(s) + 1;
16      . append(s, new) = s;
17      . append(s1, add(s2, c)) = add(append(s1, s2), c)
18      . equal(new, new);
19      . not equal(new, add(s, c));
20      . not equal(add(s, c), new);
21      . equal(add(s1, c), add(s2, c)) <=> equal(s1, s2);
22 end
```

Deduzione di proprietà

(a) $\text{append}(\text{new}, \text{add}(\text{new}, c)) = \text{add}(\text{new}, c)$

Dall'assioma 17 con $s1=s2=\text{new}$,

(1) $\text{append}(\text{new}, \text{add}(\text{new}, c)) = \text{add}(\text{append}(\text{new}, \text{new}), c);$

Dall'assioma 16, con $s=\text{new}$, $\text{append}(\text{new}, \text{new}) = \text{new}$

e sostituendo in (1) la tesi.

(b) $\text{append}(\text{new}, s) = s$

Per induzione sulla lunghezza di s :

- se $s=\text{new}$: (b) è vera per l'assioma 16
- se (b) è vera per ogni $s0$ di lunghezza L :
ogni $s1$ di lunghezza $L + 1$ sarà $\text{add}(s0, c)$ per qualche $s0$ e c , e
(1) $\text{append}(\text{new}, s1) = \text{append}(\text{new}, \text{add}(s0, c));$
per l'assioma 17 con $s1=\text{new}$ e $s2=s0$,
(1) $= \text{add}(\text{append}(\text{new}, s0), c) = \text{add}(s0, c) = s1.$

Possibili problemi di una specifica algebrica

Incompletezza: impossibilità di dimostrare proprietà desiderabili

Esempio

Si può dimostrare `not equal(add(s,'a'),add(s,'b'))`?

No. Diventa possibile sostituendo l'ultimo assioma

con:

```
equal (add (s1, c1), add (s2, c2)) <=> equal (s1,s2) /\  
equalC(c1,c2);
```

Sovraspecifica: aggiunta di assiomi che vincolano troppo le operazioni

Esempio

Se l'ultimo assioma fosse: `equal (add (s1, c1), add (s2, c2)) <=> equal (s1,s2) /\ equalC(c1,c2) not equalC(c1,'a')`;

Afferma che due stringhe sono uguali se non contengono il carattere `a`.

Possibili problemi di una specifica algebrica

Incoerenza: contraddizione fra assiomi. Si ha quando si riesce a dimostrare $\text{true}=\text{false}$

Esempio

L'assioma `equal(add(s,c),s)` introdurrebbe un'incoerenza.

Ridondanza: Presenza di assiomi non necessari in quanto derivabili

Esempio

L'assioma `append(new,s)=s` sarebbe ridondante.

Valutazione delle specifiche algebriche

Vantaggi:

- specifica formale, di più alto livello rispetto ai contratti
- si presta a verifiche statiche, manuali (come abbiamo visto) o automatizzate (tramite strumenti)
- vicine al codice dei linguaggi più dichiarativi (e, sempre più spesso, anche di quelli imperativi)
- base per testing, sia tradizionale che property-based

Svantaggi:

- non facili da leggere
- ancor meno facili da scrivere
- difficilmente applicabili per interi sistemi di dimensioni non banali

Esercizio

- Dare una specifica algebrica del tipo di dato Pila, con le operazioni consuete.
Non importa la sintassi utilizzata ma è necessario specificare in modo rigoroso
 - sort coinvolti
 - operazioni
 - assiomi
- Implementare il sort Pila nel modo più fedele possibile alla specifica (si suggerisce, per chi li conosce, Haskell o Prolog)